

C 语言错题本 · 第二批（作业单选题 1~21）

整理日期：2026-06-14

来源：超星作业截图 + 解析笔记

目录

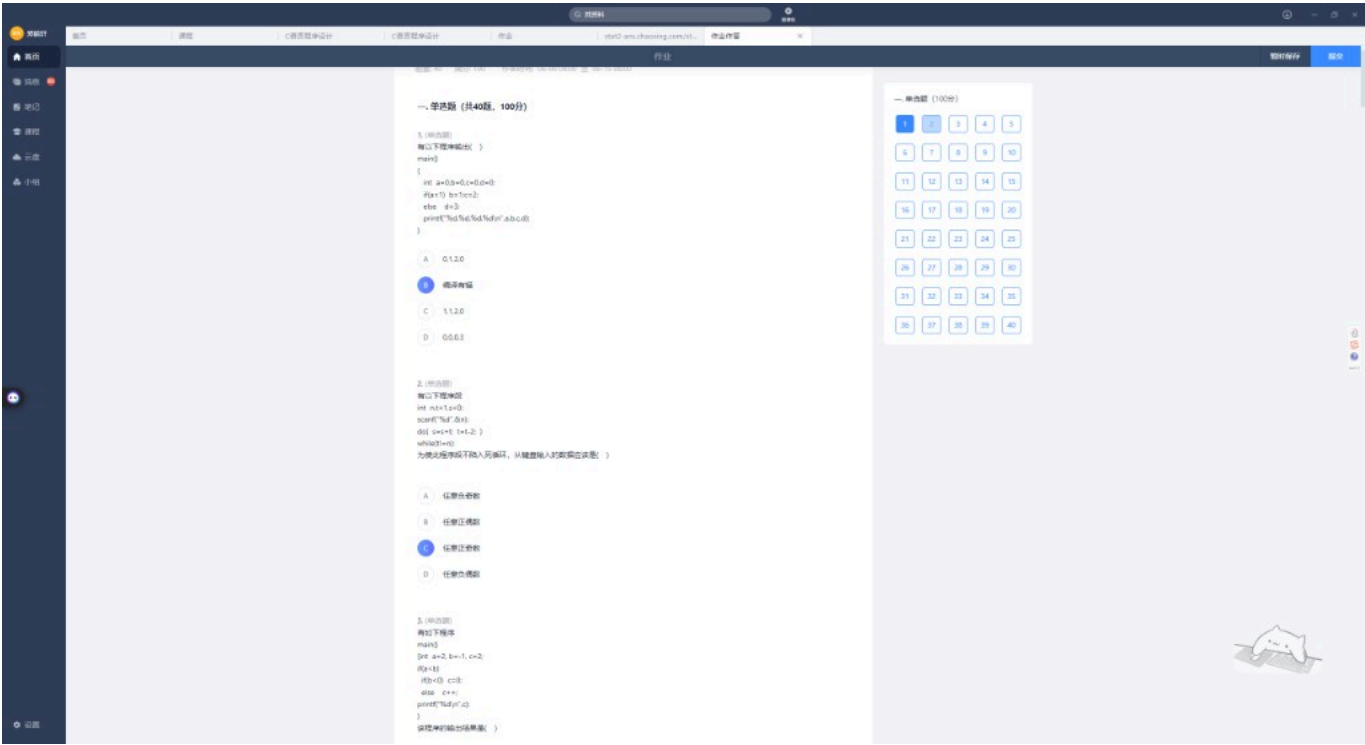
作业题目

- 第 1 题 · if-else 无花括号
- 第 2 题 · do-while 死循环
- 第 3 题 · if 嵌套跳过
- 第 4 题 · switch 穿透 + for 循环
- 第 5~7 题 · 概念与逻辑
- 第 6 题 · 转义字符综合
- 第 8 题 · 大小写互换（魔法 32）
- 第 9 题 · 混合运算
- 第 10 题 · if 只控制一条语句
- 第 14 题 · 连环比较
- 第 15 题 · switch 穿透 + for 顺序
- 第 16 题 · 逻辑与短路
- 第 17~18 题 · printf 与 for
- 第 19~21 题 · 自增与逗号、三种结构

知识点专题

- while vs do-while
 - for 循环四步规则
 - 从右向左的运算符
-

第 1 题



题目：以下程序输出（ ）

```
main()
{
    int a=0, b=0, c=0, d=0;
    if(a=1) b=1; c=2;
    else d=3;
    printf("%d, %d, %d, %d\\n", a, b, c, d);
}
```

你的答案	正确答案
B 编译有错 ✓	B 编译有错

错因：if 无 {} 只控制 b=1；，c=2；夹在中间导致 else 找不到 if。
(详见第一批错题本第 8 章)

第 2 题

题目：为使程序不陷入死循环，键盘输入的数据应当是（ ）

```
int n, t=1, s=0;
scanf("%d", &n);
do {
    s = s + t;
    t = t - 2;
} while(t != n);
```

你的答案

正确答案

C 任意正奇数

A 任意负奇数

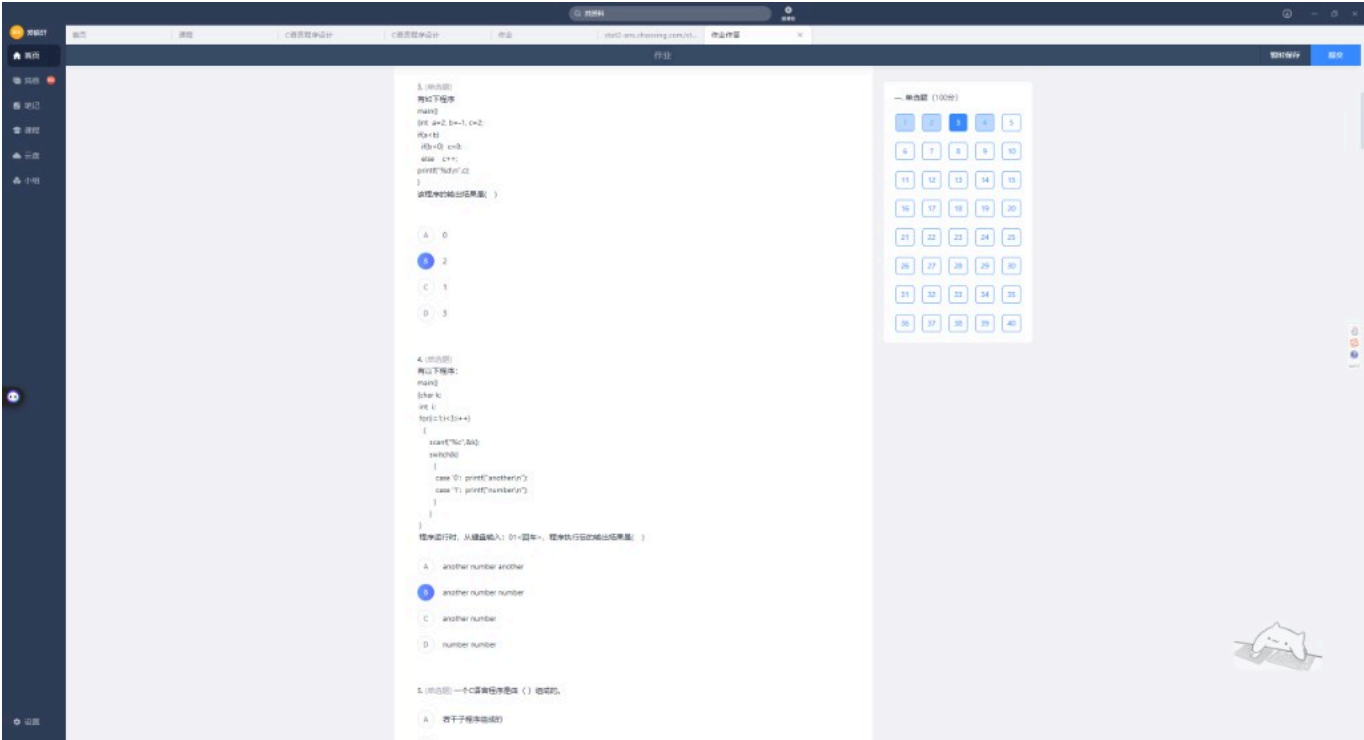
解析：

- do-while 先执行一次，t 的变化序列：1 → -1 → -3 → -5 → ...
- 循环在 while(t != n) 为假时结束，即 n 必须等于某次执行后的 t
- t 只会变成 -1、-3、-5...（负奇数）
- 若输入正奇数（如 3），t 永远不等于 3 → 死循环

⚠ 避坑指南

正奇数 1 也不行：第一次 do 后 t 就变成 -1，再也回不到 1。

第 3 题



题目：

```
main()
{
```

```
int a=2, b=-1, c=2;
if(a<b)
    if(b<0) c=0;
    else c++;
printf("%d\n", c);
}
```

正确答案

B. 2

解析：

- $a < b \rightarrow 2 < -1$ 为假，整个 `if` 块跳过
- `c` 保持初值 **2**

⚠ 避坑指南

`else` 只和最近的 `if` 配对。这里 `else c++` 属于内层 `if(b<0)`，不是外层 `if(a<b)`。

第 4 题

🏆 第 4 题解析：双重陷阱大挑战

代码核心：

c

📄 复制

```
for(i=1; i<3; i++) {  
    scanf("%c", &k);  
    switch(k) {  
        case '0': printf("another\n");  
        case '1': printf("number\n");  
    }  
}
```

输入：01<回车>（缓冲区里现在有三个字符：'0'、'1'、'\n'）

陷阱一：switch 的穿透效应 (Fall-through)

这题的 switch 里面没有 break！这意味着一旦匹配成功，代码就会像刹车失灵一样，无视后面的 case 标签一路往下执行。

陷阱二：for 循环的次数

注意看循环条件：i = 1; i < 3; i++。

这意味着循环只会执行 2 次（当 i=1 和 i=2 时）。虽然你输入了回车，但循环根本轮不到去读那个回车符就结束了。

🎬 剧情重演（程序执行过程）：

• 第 1 圈（i=1）：

- i. scanf 从传送带（缓冲区）上拿走第一个字符 '0'，赋值给 k。
- ii. 进入 switch('0')，匹配到 case '0'。
- iii. 打印 another\n（换行）。
- iv. 【穿透发生！】因为没有 break，直接掉进下一层，继续执行 case '1' 里的代码。
- v. 打印 number\n（换行）。

• 此时屏幕输出：another number

- 第 2 圈 ($i=2$) :

- i. `scanf` 从传送带上拿走第二个字符 `'1'`，赋值给 `k`。
- ii. 进入 `switch('1')`，直接跳过 `case '0'`，匹配到 `case '1'`。
- iii. 打印 `number\n` (换行)。

- 此时屏幕累加输出: `another number number`

- 第 3 圈准备 ($i=3$) : 判断 `3 < 3` 是假的，循环直接结束！传送带上剩下的那个 `<回车>` 字符被永远留在了缓冲区里，无人理睬。

最终结果完美契合选项 B!

题目：输入 `01<回车>`，输出 ()

```
main()
{
    char k;
    int i;
    for(i=1; i<3; i++)
    {
        scanf("%c", &k);
        switch(k)
        {
            case '0': printf("another\n");
            case '1': printf("number\n");
        }
    }
}
```

正确答案

B. another number number

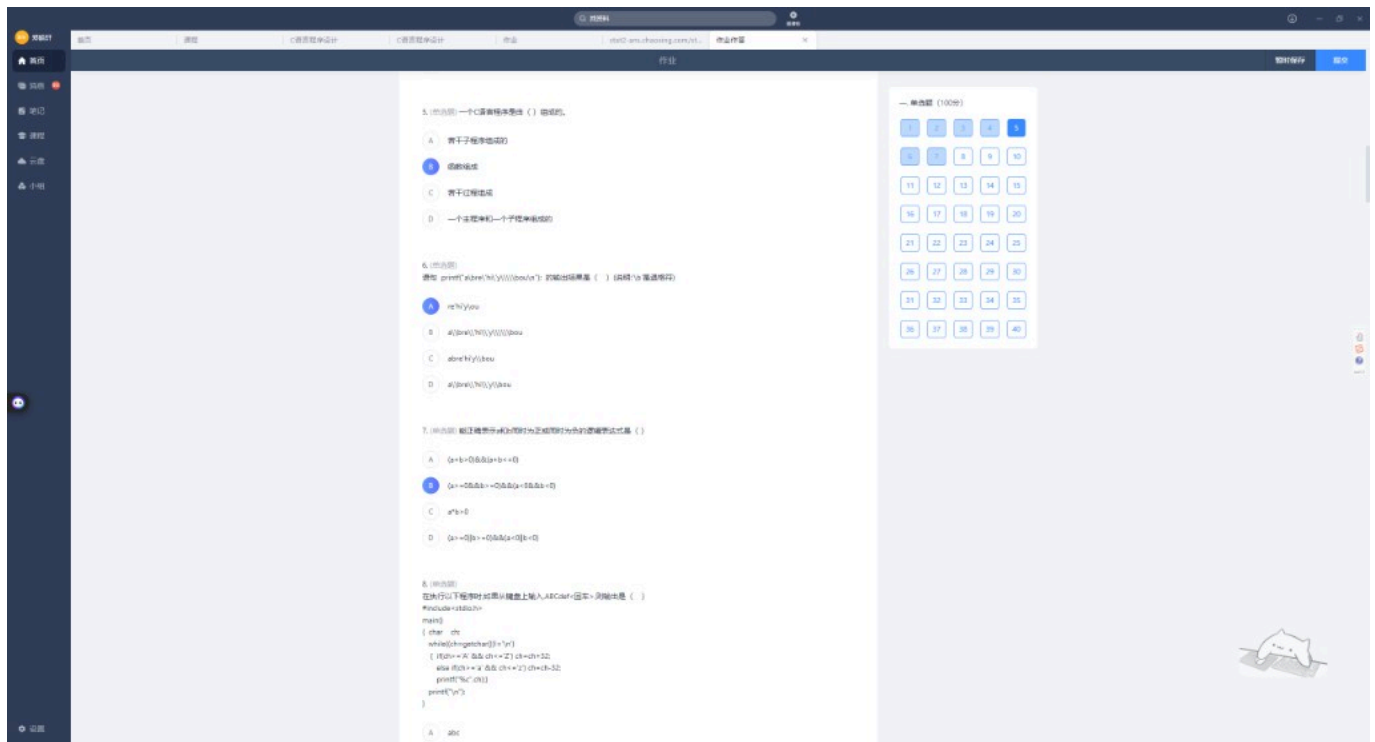
双重陷阱：

1. **switch 穿透**：没有 `break`，匹配后一路"滑下去"
2. **for 只跑 2 圈**： $i=1,2$ ，缓冲区有 `'0''1'\n` 三个字符，但只读前两个

执行过程：

圈次	读到	输出
$i=1$	<code>'0'</code>	<code>another</code> + 穿透 → <code>number</code>
$i=2$	<code>'1'</code>	<code>number</code>
$i=3$	不满足 <code>i<3</code>	结束， <code>'\n'</code> 留在缓冲区

第 5~7 题



第 5 题：C 程序由什么组成？

正确答案

B. 函数组成

第 7 题: a 和 b 同正或同负的逻辑表达式

你的答案

也正确的写法

B $(a \geq 0 \& \& b \geq 0) \vee (a < 0 \& \& b < 0)$ ✓ C $a * b > 0$ (同号且均不为 0)

第 6 题

题目: `printf("a\bre\'hi\'y\\bou\n");` 输出 () (`\b` 是退格符)

你的答案

正确答案

A re'hi'you ✓ **A. re'hi'you**

逐步拆解：

- a → 打出 a
- \b → 光标退回，覆盖 a
- re'hi'y → 打出 re'hi'y
- \\ → 打出一个反斜杠 \
- \b → 退格，覆盖掉刚打的 \

bou → 打出 bou
\n → 换行

最终屏幕：re'hi'you

第 8 题

第 8 题：大小写字母的“魔法数字 32”

这道题截图里没看到你选的答案，咱们直接来拆解代码：

c

复制

```
while((ch=getchar())!='\n') {
    if(ch>='A' && ch<='Z') ch=ch+32;
    else if(ch>='a' && ch<='z') ch=ch-32;
    printf("%c",ch);
}
```

输入： ABCdef<回车>

核心知识点：ASCII 码的秘密

在 C 语言里，字符其实就是数字（ASCII 码）。

- 大写字母 **A** 的编号是 65。
- 小写字母 **a** 的编号是 97。
- 它们之间刚好相差 **32**！

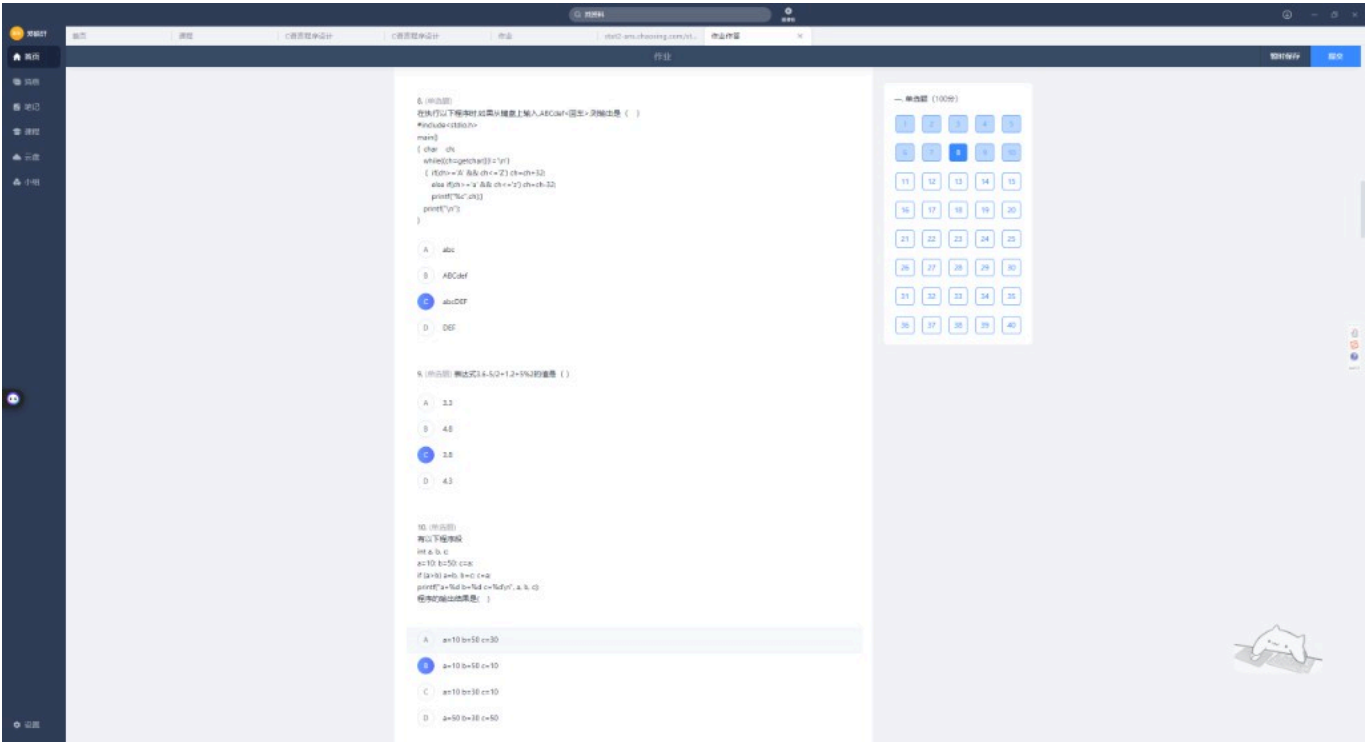
代码逻辑：

1. `getchar()` 每次从输入里拿一个字符，直到遇到回车 `\n` 结束。
2. `if(ch>='A' && ch<='Z') ch=ch+32;` 🟡 如果是大写字母，加上 32，就变成了小写字母。
3. `else if(ch>='a' && ch<='z') ch=ch-32;` 🟡 如果是小写字母，减去 32，就变成了大写字母。

所以，这段代码的功能就是：大小写互换！

你输入了 `ABCdef`，经过转换后，输出的结果必然是：

🟡 `abcDEF`



题目：输入 ABCdef<回车>，输出（ ）

```
#include <stdio.h>
main()
{
    char ch;
    while((ch=getchar())!='\n')
    {
        if(ch>='A' && ch<='Z') ch=ch+32;
        else if(ch>='a' && ch<='z') ch=ch-32;
        printf("%c", ch);
    }
    printf("\n");
}
```

你的答案

正确答案

C abcDEF ✓

C. abcDEF

魔法数字 32：

- 'A' = 65，'a' = 97，差值 32
- 大写 +32 → 小写；小写 -32 → 大写
- 本题是大小写互换

第 9 题

题目：3.6 - 5/2 + 1.2 + 5%2 的值是（ ）

你的答案	正确答案
------	------

C 2.8 ✓	C. 2.8
---------	---------------

计算：

```
5/2 = 2      (整数除法)
5%2 = 1
3.6 - 2 + 1.2 + 1 = 2.8
```

第 10 题

题目：

```
int a, b, c;
a=10; b=50; c=a;
if (a>b) a=b; b=c; c=a;
printf("a=%d b=%d c=%d\n", a, b, c);
```

你的答案	正确答案
------	------

B a=10 b=50 c=10	a=10 b=10 c=10
------------------	-----------------------

解析：

```
if (a>b) a=b;    // if 只控制这一句，a>b 假，跳过
b=c;            // 独立执行，b=10
c=a;            // 独立执行，c=10
```

⚠ 避坑指南

不要误以为 `b=c; c=a;` 也在 `if` 里面。无 `{ }` 时只有紧跟的一条语句受 `if` 控制。

第 14 题

[illegible]

🚩 第 14 题：连环比较的“视觉欺骗”

你选了 A（编译时有错），但这次出题老师把你给骗了，正确答案其实是 D（3）。

你可能会想：`if(a>b>c)` 这种连着写大于号的语法，在 C 语言里难道不报错吗？

答案是：绝对不报错！它是合法的！

但在 C 语言里，它的计算逻辑和我们在数学里的直觉完全不一样。你可以把下面这段加进你的错题本：

💡 避坑指南：连续比较表达式 `a > b > c`

在 Python 或者数学里，`5 > 4 > 3` 是成立的。

但在 C 语言里，运算符是从左向右算的，它会变成这样：

1. 先算 `a > b`，也就是 `5 > 4`，结果为真（也就是 1）。
2. 然后用刚才算出来的 1，去和 `c` 比较，也就是 `1 > 3`。
3. `1 > 3` 显然是假（也就是 0）！结论：`if(a>b>c)` 在这道题里不仅没语法错误，而且它的结果是 0（假）！

顺着代码往下走：

1. `if(a>b>c)` 是假，跳过。
2. 来到 `else if((c-1)>=d)==1)`：
 - `c-1` 是 `3-1=2`。
 - `2 >= d` 就是 `2 >= 2`，结果为真（1）。
 - `1 == 1`，结果为真！
3. 执行 `printf("%d\n", d+1);`，也就是打印 2+1。
4. 最终输出结果：3。

题目：

```
main()
{
    int a=5, b=4, c=3, d=2;
    if(a>b>c) printf("%d\n", d);
    else if((c-1)>=d)==1) printf("%d\n", d+1);
    else printf("%d\n", d+2);
}
```

你的答案 正确答案

A 编译有错 D. 3

连环比较陷阱：`a>b>c` 不是语法错误！

a>b>c → (a>b)>c → (5>4)>3 → 1>3 → 0 (假)

走 else if:

(c-1>=d)==1 → (3-1>=2)==1 → (2>=2)==1 → 1==1 → 真
输出 d+1 = 3

第 15 题

题目:

```
main()
{
    int i;
    for(i=0; i<3; i++)
        switch(i)
        {
            case 0: printf("%d", i);
            case 2: printf("%d", i);
            default: printf("%d", i);
        }
}
```

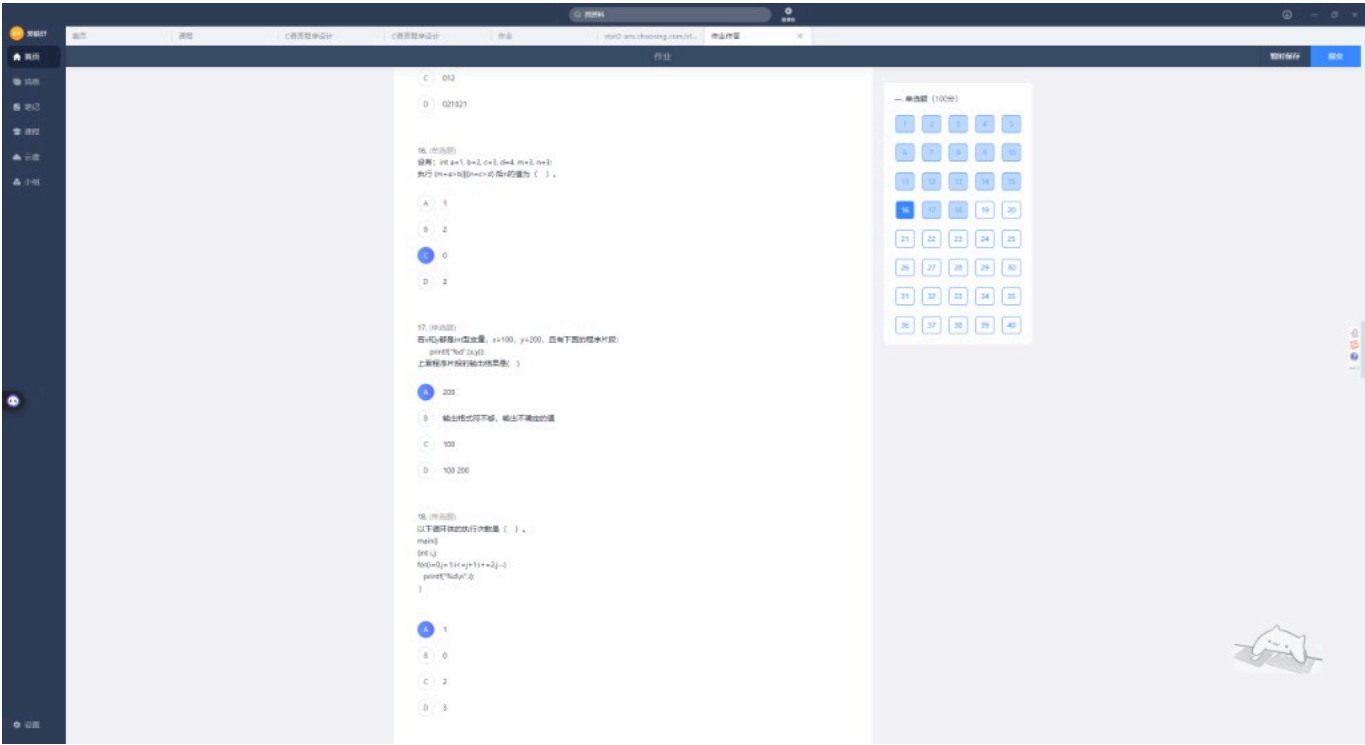
你的答案 正确答案

A 000122 ✓ A. 000122

执行过程 (注意 for 四步规则 + switch 穿透):

i	switch 输出	说明
0	000	case 0→case 2→default 全穿透
1	1	只进 default
2	22	case 2→default 穿透

第 16 题



题目：int a=1,b=2,c=3,d=4,m=3,n=1; 执行 (m=a>b)&&(n=c>d) 后 n 的值 ()

你的答案 正确答案

C 0 A. 1 (n 保持初值)

解析：

m = a>b → m = 0 (假)
&& 短路：左边为假，右边 n=c>d 根本不执行
n 保持初值 1

⚠ 避坑指南

&& 左边为假时，右边不会执行，不要被 n=c>d 的写法迷惑。

第 17~18 题

第 17 题

printf("%d", x, y); — 只有一个 %d，只输出 x = 100

你的答案 正确答案

A 200 C. 100

第 18 题

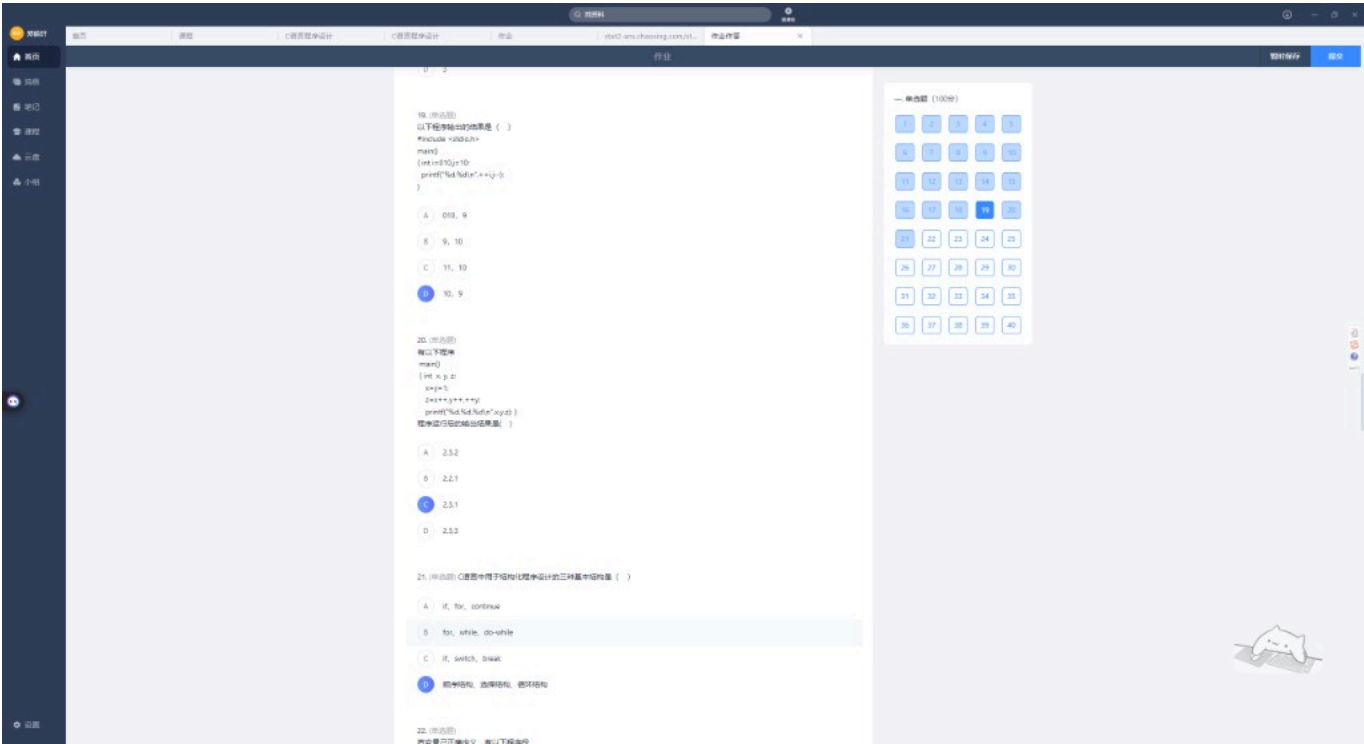
```
for(i=0, j=1; i<=j+1; i++, j--)  
    printf("%d\n", i);
```

你的答案 正确答案

A 1 C. 2

轮次	i	j	i<=j+1	输出
1	0	1	0<=2 真	0
2	1	0	1<=1 真	1
3	2	-1	2<=0 假	结束

第 19~21 题



第 19 题

```
int i=10, j=10;  
printf("%d, %d\n", ++i, j--);
```

你的答案 常见答案

D 10, 9 C. 11, 10

++i 先加后用 → 11; j-- 先用后减 → 10

第 20 题

```
x=y=1;  
z=x++, y++, ++y;  
printf("%d, %d, %d\n", x, y, z);
```

你的答案	正确答案
------	------

C 2, 3, 1 ✓	C. 2, 3, 1
-------------	-------------------

逗号表达式: $(z=x++)$, $(y++)$, $(++y) \rightarrow z=1, x=2, y=3$

第 21 题

正确答案

D. 顺序结构、选择结构、循环结构

专题: while vs do-while

1. 普通的 while 循环 (乖宝宝)

- **逻辑**: 先判断条件, 条件成立才执行。
- **比喻**: 就像去游乐园坐过山车。检票员先量你的身高 (判断条件), 如果身高够了, 才让你进去坐 (执行代码)。如果一开始身高就不够, 你一次都坐不了。

2. do-while 循环 (莽夫)

- **逻辑**: 不管三七二十一, 先强行执行一次代码, 执行完了再去判断条件。如果条件成立, 就再来一次; 如果不成立, 就拍拍屁股走人。
- **比喻**: 就像超市里的免费试吃。推销员不管你带没带钱, 先往你嘴里塞一块香肠让你吃 (do 先执行一次)。你吃完之后, 他再问你“好吃吗? 买不买?” (判断 while 条件)。

代码直观对比

假设我们有一个变量 `x = 10`, 我们想在 `x < 5` 的时候打印数字。

如果是 `while` :

c 复制

```
int x = 10;
while (x < 5) {
    printf("hello\n");
}
// 结果: 什么都不会打印。因为一开始 10 < 5 就是假的, 门都进不去。
```

如果是 `do-while` :

c 复制

```
int x = 10;
do {
    printf("hello\n");
} while (x < 5);
// 结果: 屏幕上会打印出一个 hello!
// 解释: 电脑先执行了 do 里面的打印, 打印完才去看了眼条件, 发现 10 < 5 是假的, 这才停下来。
```

类型	昵称	逻辑
while	乖宝宝	先判断, 条件成立才执行
do-while	莽夫	先执行一次, 再判断条件

经典对比 (`x=10`, 条件 `x<5`):

```
// while: 一次都不执行
while (x < 5) printf("hello\n");
```

```
// do-while: 至少执行一次, 打印一个 hello
do { printf("hello\n"); } while (x < 5);
```

专题：for 循环四步规则

你是不是以为，程序看到 `for(i=0; i<3; i++)` 时，会立刻执行后面的 `i++` 把 `i` 变成 1，然后再去执行 `switch` 里的代码？

其实不是的！这正是 `for` 循环最容易骗人的地方。

在 C 语言里，`for` 循环的执行顺序有着极其严格的“四步走”规则。咱们把 `for(i=0; i<3; i++)` 拆解开来看，它其实是一个“操场跑圈”的过程：

🏃 `for` 循环的“跑圈”规则：

1. 第一步：站在起跑线（初始化）👉 `i=0`
 - 这一步只在最开始执行一次。此时 `i` 就是 0。
2. 第二步：裁判看你有没有犯规（条件判断）👉 `i<3`
 - 裁判看看现在的 `i` 是不是小于 3。0 小于 3，没问题，允许起跑！
3. 第三步：跑一圈（执行循环体代码）👉 `switch(i)`
 - 注意！这个时候 `i` 依然是 0！
 - 程序带着 `i=0` 进入了 `switch`，执行了打印 `000` 的操作。
4. 第四步：跑完一圈，喝口水调整一下（执行增量操作）👉 `i++`
 - 划重点：`i++` 是在循环体（大括号里的代码）全部执行完之后，才最后执行的！
 - 直到这个时候，`i` 才从 0 变成了 1。

`for(i=0; i<3; i++)` 的严格顺序：

1. 初始化 `i=0`（只执行一次）
2. 判断条件 `i<3`？成立才进入
3. 执行循环体（此时 `i` 还没自增！）
4. 自增 `i++`（循环体全部跑完才执行）

⚠️ 避坑指南

`i++` 不是在判断之后立刻执行，而是在整个循环体跑完之后才执行。

专题：从右向左的运算符

← BACK C 语言里的三大“逆行者”（从右向左算）

1. 赋值运算符（=, +=, -=, *= 等）

这是最常见、也最符合我们直觉的从右向左算的符号。

例子：连环赋值

c

复制

```
int a, b, c;  
a = b = c = 10;
```

电脑是怎么算的？

它绝对不会先算 `a = b`（因为这时候 `b` 还是个垃圾值）。

它会从最右边开始：

1. 先算 `c = 10`。
2. 然后把 `c = 10` 的结果（也就是 10）赋给 `b`，变成 `b = 10`。
3. 最后把 `b = 10` 的结果赋给 `a`，变成 `a = 10`。

2. 单目运算符（只带一个“小弟”的符号）

所谓单目，就是这个符号只需要一个数字就能工作。比如：`!`（逻辑非）、`++`、`--`、`-`（负号）、`&`（取地址）。

例子：多个单目符号挤在一起

c

复制

```
int a = 5;  
int b = !-a;
```

电脑是怎么算的？

它会从靠近变量的右边开始，一层层往左剥洋葱：

1. 先算右边的 `-a`，把 5 变成 `-5`。
2. 再算左边的 `!`（逻辑非）。因为 `-5` 是非 0（真），取反之后就变成了假（0）。
3. 最后把 0 赋给 `b`。

3. 三目运算符 (条件运算符 ? :)

这是 C 语言里唯一——一个需要三个操作数的符号，当它们连在一起时，也是从右往左算的。

例子：嵌套的三目运算

c

复制

```
int a = -1, b = 5;
int result = a > 0 ? 1 : b > 0 ? 2 : 3;
```

电脑是怎么算的？

它会从右边开始加括号，变成这样： `a > 0 ? 1 : (b > 0 ? 2 : 3)`

- 1. 先看右边那一坨 `(b > 0 ? 2 : 3)`，因为 `5 > 0` 为真，所以右边这坨的结果是 `2`。
- 2. 再看整体 `a > 0 ? 1 : 2`，因为 `-1 > 0` 为假，所以最终结果选冒号后面的 `2`。

C 语言三大"逆行者"（从右向左算）：

1. 赋值运算符

```
a = b = c = 10; // 先 c=10, 再 b=10, 再 a=10
```

2. 单目运算符

```
int a = 5;
int b = !-a; // 先算 -a=-5, 再 !(-5)=0
```

3. 三目运算符（嵌套时）

```
int a=-1, b=5;
int result = a>0 ? 1 : b>0 ? 2 : 3;
// 等价于 a>0 ? 1 : (b>0 ? 2 : 3) → 2
```

本批易错点速记

题号	易错点	一句话
1	if-else 无花括号	else 中间不能夹语句
2	do-while + t 序列	输入应为负奇数，不是正奇数

题号	易错点	一句话
4	switch 无 break	穿透 + for 只跑 2 圈
10	if 控制范围	无 {} 只控制一条语句
14	a>b>c	合法语法，按 (a>b)>c 算
16	&& 短路	左边假，右边不执行
17	printf 格式符	几个 % 输出几个值
18	for 多变量	注意 i 和 j 同步变化
19	自增顺序	++i 先加，j-- 先用

附录：原始截图索引

文件名	内容
01_作业题1-3.png	第 1~3 题截图
02_while与do-while对比.png	while vs do-while 专题
03_第4题解析_第1圈.png	第 4 题第 1 圈执行
04_第4题解析_第2圈.png	第 4 题第 2 圈执行
05_作业题3-5.png	第 3~5 题截图
06_作业题5-8.png	第 5~8 题截图
07_作业题8-10.png	第 8~10 题截图
08_第8题_魔法数字32.png	第 8 题 ASCII 解析
09_作业题14-16.png	第 14~16 题截图
10_第14题_连环比较.png	第 14 题连环比较
11_for循环跑圈规则.png	for 循环四步规则
12_从右向左运算符.png	赋值、单目运算符
13_三目运算符嵌套.png	三目运算符嵌套
14_作业题16-18.png	第 16~18 题截图
15_作业题19-21.png	第 19~21 题截图

下一批照片发来后，存入 `images/第三批/` 并追加。